

Toward a Witnessability Model for AI and Software Execution Systems

A Boundary-Based Framework for Classifying Execution Evidence

Mikhail Anatolievich Sergeev
Independent Researcher
ORCID 0009-0001-6443-855X

Vladimir Ikher
Independent Researcher
ORCID 0009-0000-0084-704X

Version 1.0 · June 2026

Abstract

Modern AI and software systems increasingly rely on autonomous execution, orchestration frameworks, workflow engines, agent runtimes, tool-calling protocols, serverless execution environments, and governance layers. These systems generate many forms of evidence: logs, traces, metrics, audit records, policy decisions, workflow histories, tool-call records, software attestations, provenance statements, and compliance reports. However, these artifacts often answer different questions and are frequently conflated.

This paper introduces the *Witnessability Model*, a boundary-based framework for classifying the strength of execution-related evidence available for actions performed by software and AI systems. The model distinguishes six execution-evidence levels: observation, intent, invocation, execution, provenance linkage, and independent verification. The paper’s central result, the Boundary Ceiling Principle, states that a witness cannot honestly claim an execution-evidence level above the strongest execution-relevant boundary it controls or can independently verify.

The paper further proposes a *Witness Boundary Taxonomy* and applies the model to contemporary systems including observability frameworks, CI/CD platforms, workflow engines, AI agent frameworks, tool-calling protocols, automation platforms, serverless runtimes, coding agents, software supply-chain attestation systems, and trusted execution mechanisms.

The analysis suggests that execution ownership, rather than the number of integrations, policies, traces, or attestations, is the primary determinant of attainable witnessability. The model clarifies the distinction between seeing an event, approving an action, invoking a tool, owning execution, linking execution to provenance, and enabling independent verification.

Because these levels carry different evidentiary weight, conflating them creates operational, legal, and governance risk in audit, liability, and AI-governance settings.

Keywords: witnessability, AI agents, execution evidence, provenance, attestations, observability, software supply chain, runtime verification, execution ownership, agent runtimes, AI governance, auditability, operational risk, independent verification, software execution.

1 Introduction

Software systems increasingly act through automated pipelines, workflow engines, AI agents, orchestration platforms, serverless runtimes, and tool-calling frameworks. In many environments, humans no longer execute each action directly. Instead, they define goals, prompts, policies, workflows, permissions, deployment rules, or constraints, while software systems perform actions across chains of components and trust domains.

This shift creates a growing evidence problem. Modern systems can often produce logs, metrics, traces, audit records, policy decisions, tool-call records, workflow histories, execution

traces, provenance attestations, software bills of materials, and compliance reports. Yet these artifacts do not all prove the same thing.

A policy system may prove that an action was approved. An observability system may show that a request occurred. A workflow engine may show that a task was scheduled. A supply-chain attestation may prove how an artifact was built. A runtime verification monitor may evaluate whether a trace satisfies a formal property. None of these necessarily proves, by itself, what action actually executed, under which boundary, with which inputs, with which outputs, and with what degree of independent verifiability.

1.1 Problem Statement

Evidence artifacts in modern AI and software systems are often conflated under broad terms such as “auditability,” “traceability,” “observability,” “provenance,” “compliance,” or “verification.” This makes it difficult to evaluate what a system can honestly claim about execution.

For example, a governance system may prove that a decision was approved, but not that the approved action executed. An observability platform may record that a request occurred, but not own the execution environment that handled it. A tool-calling protocol may mediate invocation, but not control the tool’s internal execution. A supply-chain attestation may link software to its build process, but not prove a later runtime action. A workflow engine may own task scheduling while delegating execution to external services. An AI agent may produce a reasoning trace, but not prove that an externally invoked side effect occurred as claimed.

In institutional settings—audit, liability, AI governance, procurement, and post-incident reconstruction—this ambiguity is itself a source of operational, legal, and governance risk: an organization may be able to show that an action was approved or logged, yet be unable to establish what actually executed, under which boundary, and with what independently verifiable evidence.

1.2 Research Question

This paper asks:

What execution-evidence levels can be produced by software and AI systems, and what determines the maximum level that can be honestly claimed?

A secondary question follows:

How can systems be classified according to the boundary they control and the resulting ceiling of witnessability?

1.3 Contribution

This paper contributes:

1. The *Witnessability Model*, which classifies execution-evidence levels from WL0 to WL5.
2. The *Boundary Ceiling Principle*, which states that a witness cannot honestly claim evidence above the strongest execution-relevant boundary it controls or can independently verify.
3. A *Witness Boundary Taxonomy*, distinguishing observation, intent, invocation, execution, provenance, and verification boundaries.
4. A comparative evaluation framework for analyzing observability tools, CI/CD platforms, workflow engines, AI agent frameworks, tool-calling protocols, serverless runtimes, coding agents, and supply-chain attestation systems.

5. A conceptual separation between observability, governance, provenance, runtime verification, and witnessability.

This paper does not claim to introduce runtime evidence, provenance, receipts, or execution tracing as new categories; its contribution is the boundary-based ceiling invariant that constrains the maximum honest execution-evidence claim a witness can make. It is a conceptual framework rather than a system or implementation: like other vendor-neutral assurance and provenance frameworks, its contribution is definitional and classificatory, and it is intended to be evaluated on those terms.

1.4 What Is New

The model is not a claim that common evidence artifacts are new. This paper does not claim that logs are new, that provenance is new, that attestations are new, or that verification is new. It also does not claim that observability, auditability, provenance, or runtime verification are reducible to witnessability.

The claim is narrower. This paper introduces a boundary-based ceiling on execution-evidence claims; a formal distinction between observation, invocation, and execution ownership; and a claim-strength classification that is independent of artifact type. A log, trace, attestation, workflow history, or verification report may support different execution-evidence claims depending on which boundary produced it and which boundary can verify it.

	Observability	Auditability	Provenance	Runtime Verification	Witnessability
Primary question	What can be seen or measured?	Can actions and decisions be reviewed later?	Where did an artifact or result come from?	Does an execution trace satisfy a property?	What execution-evidence claim can a witness honestly support?
Evidence type	Logs, metrics, traces, events.	Records, controls, approvals, audit trails.	Source, build, artifact, dependency, or origin links.	Traces plus formal or executable properties.	Boundary-grounded claims about action evidence.
Execution ownership required?	No.	No.	Not by itself.	Not by itself.	Required for WL3 execution evidence and above.
Independent verification required?	Not inherent.	Sometimes, depending on audit regime.	Often useful for origin claims.	Not inherent; monitors may be internal.	Required for WL5.
Honest claim ceiling?	Not defined.	Not defined.	Supports origin/provenance claims, not runtime action by itself.	Evaluates properties over available evidence.	Explicitly defined by the strongest controlled or independently verified boundary.

Table 1: Witnessability compared with adjacent evidence concepts.

1.5 Central Thesis

The central thesis of this paper is:

The maximum strength of an execution-evidence claim is determined not by the number of logs, integrations, policies, traces, or attestations a system produces, but by the execution-relevant boundary controlled by the witness.

2 Motivating Examples

2.1 Governance Approval Without Execution Proof

An AI governance layer may evaluate a proposed tool call and record that the action was allowed by policy. This record may be valuable for compliance. It may show intent, authorization, and policy alignment. However, such a record does not prove that the action actually executed.

2.2 Trace Visibility Without Execution Ownership

A distributed tracing system may record that a request crossed a service boundary. It may show timing, service identity, latency, status codes, and related spans. However, unless the tracing system controls the execution boundary, the trace remains an observation of behavior rather than proof of owned execution.

2.3 Tool Invocation Without Tool Execution Proof

An AI agent may call an external tool through a standardized protocol. The agent runtime may record the tool name, parameters, response, and timestamp. This may prove that an invocation occurred across a controlled boundary. But if the actual tool executes in an external system, the agent runtime may not be able to prove how that tool executed internally.

2.4 Build Provenance Without Runtime Proof

A software artifact may have strong build provenance, signed attestations, and verifiable source linkage. This may establish where the artifact came from and how it was built. However, build provenance does not automatically prove that a specific runtime action was later executed by that artifact under a specific environment.

2.5 Execution Ownership Without Independent Verification

A workflow engine, serverless runtime, or CI/CD runner may own the execution process. It may capture command, environment, exit code, outputs, and result digests. This can provide strong execution evidence. But if all evidence is produced and stored by the same operator, independent verification may still require additional mechanisms such as signed receipts, transparency logs, reproducible checks, remote attestation, external witnesses, or third-party verification.

3 Related Work

3.1 Observability

Observability systems collect, process, export, and correlate telemetry such as logs, metrics, traces, events, and resource metadata. OpenTelemetry is a leading vendor-neutral observability framework that supports the generation, collection, and export of telemetry data such as traces, metrics, and logs [1, 2]. Observability systems are essential for debugging, monitoring, incident response, performance analysis, and operational visibility.

However, observability is primarily concerned with what can be seen, measured, and correlated. It does not, by itself, define the maximum execution-evidence claim that a system can honestly make.

3.2 Runtime Verification

Runtime verification studies the dynamic analysis of execution traces against formal specifications [3, 4]. It typically involves instrumentation, trace generation, monitor synthesis, and the evaluation of traces against generated monitors or formalized requirements.

Witnessability addresses a different but complementary question. Runtime verification asks whether an execution satisfies a specified property. Witnessability asks what kind of execution-related evidence exists before a domain-specific property is evaluated.

Terminology: witnessability. The term *witnessability* has prior use in programming languages and formal methods, notably in Ding and Zhang’s work on the witnessability of undecidable problems [5], where it concerns undecidable problems and their decidable approximations. Our use is distinct and operational: we use *witnessability* to refer to the strength of execution-evidence claims a witness can honestly support about runtime actions, given the boundary it controls.

3.3 Software Supply-Chain Security

Software supply-chain security frameworks such as SLSA, in-toto, and Sigstore focus on artifact integrity, build provenance, signing, identity, and attestations. SLSA defines provenance as verifiable information about where, when, and how a software artifact was produced [6, 7]. The in-toto Attestation Framework provides a specification for generating verifiable claims about aspects of how software is produced [8, 9]. GitHub Actions artifact attestations establish build provenance for binaries and container images [10, 11].

These systems are highly relevant to WL4 of the Witnessability Model, where runtime execution is linked to verifiable origin, source, build, artifact, configuration, or deployment materials. However, build-time provenance and runtime execution evidence are not identical.

3.4 Runtime Governance and Constitutional Frameworks

Adjacent governance architectures address authorization, constitutional constraints, delegation, and audit reconstructability for deployed AI systems. Sergeev [12] proposes a constitutional governance framework and identifies the absence of standardized, provider-independent verification infrastructure as an open limitation. Tallam [13] presents a runtime governance architecture organized around composite principals and an audit substrate for evidence reconstructability. These frameworks are complementary to the present work: they govern or reconstruct what agents may do, whereas the Witnessability Model addresses what can be proven about what they did—the execution-evidence level a witness can honestly support, bounded by the boundary it controls, independent of the trust tier of the record source.

3.5 Agent Provenance and Execution Tracing

Recent work on LLM-agent provenance studies how agent executions can be traced through tool calls, retrieved evidence, memory, intermediate claims, observations, actions, and final answers. Wang et al. provide a systematic review and conceptual framework for evidence tracing and execution provenance in LLM agents [14]. Other work explores structured reasoning provenance for autonomous AI agents [15], dynamic agent observability [16], bounding LLM agents via execution provenance [17], aligning authorization with execution provenance [18], and intent-to-execution integrity [19].

This work is directly adjacent to witnessability. It addresses how agent behavior unfolds and how evidence supports outputs or actions. The Witnessability Model adds a boundary-based claim ceiling: it asks what level of execution-related claim can honestly be made given the boundary controlled by the witness.

Work	Primary contribution	Explicit boundary-ceiling model?	Separates observe / invoke / execute?
Wang et al. [14]	Systematic review of evidence tracing / execution provenance in LLM agents	Partial	Partial
Vispute [15]	Structured reasoning provenance	Not explicit	No
AuthGraph [18]	Aligns authorization with execution provenance	Not primary focus	Partial
Sequeira et al. [17]	Runtime defense that bounds benign agent execution using action-sequence structure, function-argument provenance, allowlist checks, and an LLM judge	Not primary focus	Partial
Witnessability Model (this work)	Boundary-based ceiling on the level of execution-evidence claim	Yes (Boundary Ceiling, WL0–WL5)	Yes

Table 2: Agent-provenance / execution-tracing work compared with the Witnessability Model.

3.6 Signed Agent Receipts and an Emerging Evidence Industry

A nascent line of work focuses specifically on signed, independently checkable records of individual agent actions. Figuera’s *Notarized Agents* proposes receiver-attested, confidential receipts for AI-agent actions, in which the receiving party co-signs the evidence and a witness-cosigned log records the interaction under an adversary model that admits compromise of both the agent and its operator [20]. Cordon introduces semantic transactions for tool-using LLM agents, bounding tool execution within transactional units so that an agent’s externally visible effects can be scoped, committed, or rolled back [21]. Alongside such academic efforts, practitioner and industry work is beginning to explore agent receipts, signed attestations, and execution-evidence records for per-action accountability of autonomous systems. This activity is directly adjacent to witnessability: each effort strengthens the integrity, confidentiality, or scoping of per-action records. The Witnessability Model is complementary rather than competing: it does not propose another receipt format but classifies, for any such record, the execution-evidence level that the producing boundary can honestly support.

3.7 Workflow, Durable Execution, and Replay

Workflow engines orchestrate long-running processes, retries, state transitions, and activity execution. Temporal documents workflow event histories and durable execution through replay mechanisms [22, 23]. LangGraph provides persistence through checkpoints, enabling state snapshots, replay, human-in-the-loop workflows, time travel debugging, and fault-tolerant execution [24, 25]. Airflow defines tasks as units of execution arranged in DAGs and executed through configured executors [26, 27].

These systems may produce histories that are stronger than ordinary logs, especially when they own scheduling and activity execution. However, workflow witnessability depends on the boundary actually controlled by the workflow system.

3.8 Tool-Calling Protocols and Agent SDKs

The Model Context Protocol is an open standard for connecting AI applications to external systems, data sources, tools, and workflows [28, 29]. OpenAI’s Agents SDK includes built-in tracing of LLM generations, tool calls, handoffs, guardrails, and custom events [30]. AutoGen is a framework for multi-agent AI applications that can act autonomously or with humans [31, 32].

These frameworks are important because they often blur planning, invocation, tool execution, and observation. Their witnessability depends on whether they only orchestrate invocations or also own the execution boundary.

3.9 Execution-Grounded Coding Agents

OpenHands describes software agents that interact with developer environments by writing code, using command lines, browsing, and executing code in sandboxed environments [33, 34]. AgentForge introduces execution-grounded verification through mandatory sandboxed execution before code propagation [35]. These systems demonstrate the increasing importance of controlled execution boundaries in AI-assisted software engineering.

3.10 Serverless, Isolates, and Trusted Execution

Cloudflare Workers run code in V8 isolates and document a security model based on tenant isolation using V8 isolates rather than conventional processes or VMs [36, 37]. Trusted execution environments and remote attestation may strengthen witnessability by providing externally verifiable claims about execution environment identity and integrity [38, 39, 40]. However, attestation of the environment must still be connected to action-level evidence.

4 Definitions

Witness. A witness is a system, component, runtime, service, process, organization, or verifier that records or verifies evidence about an action.

Action. An action is an operation performed by a software or AI system that changes state, produces output, invokes a tool, calls an API, executes code, modifies data, sends a message, deploys an artifact, or otherwise affects an environment.

Evidence artifact. An evidence artifact is any record or object used to support a claim about an action.

Boundary. A boundary is a control surface across which an action is observed, approved, invoked, executed, linked to provenance, or verified.

Execution ownership. Execution ownership is the capability to run, constrain, capture, or mediate the actual execution of an action within a controlled runtime, process, worker, activity, container, isolate, virtual machine, sandbox, or equivalent execution environment.

Operationally, execution ownership is present when the witness can satisfy most of the following criteria for the action being claimed:

1. initiate or authorize the execution within a controlled runtime;
2. constrain execution through policy, sandboxing, scheduling, resource limits, or equivalent controls;
3. observe execution internals rather than only boundary-crossing events;
4. bind outputs, exit status, side effects, or result digests to the runtime that produced them; and
5. preserve execution evidence with sufficient integrity to support later reconstruction.

A witness need not satisfy every criterion in every deployment, but a claim of execution ownership becomes weak when most of these capabilities are absent.

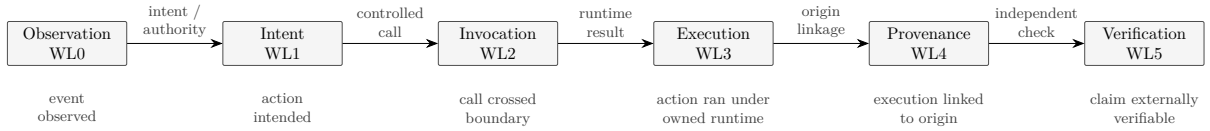


Figure 1: Evidence path from observation to independent verification. Each transition adds a new supportable claim only when the witness controls or independently verifies the corresponding boundary.

Witnessability. Witnessability is the maximum strength of an execution-related evidence claim that a witness can honestly make, given the boundary it controls and the verification mechanisms available.

4.1 System Ceiling and Event Level

The Witnessability Model defines six execution-evidence levels, denoted WL0–WL5.

A system has a *witnessability ceiling*: the maximum level it can generally support under its architecture and controlled boundary. A specific event has an *achieved execution-evidence level*: the level actually supported by the evidence captured for that event. A system may have a WL3 ceiling while a specific event only achieves WL1 if execution results were not captured.

4.2 Claim Grammar

A complete execution-evidence claim may be represented as:

Witness W claims that Actor A performed or attempted Action X under Boundary B at Time T with Input I , producing Output O , under Environment E , linked to Origin/Provenance P , and verifiable by Verifier V , under Trust Assumptions S .

Different execution-evidence levels correspond to different subsets of this claim being supportable.

Level	Honestly supportable subset of the claim grammar
WL0	Actor A , Action X , Time T as <i>observed</i> (observation boundary).
WL1	Adds intent / authority for X (intent / policy boundary).
WL2	Adds Input I across a controlled invocation boundary B .
WL3	Adds Output O and Environment E under an owned execution boundary B .
WL4	Adds Origin / Provenance P .
WL5	Adds an independent Verifier V ; weakens reliance on Trust Assumptions S .

Table 3: Claim-grammar subsets honestly supportable at each execution-evidence level; each level is cumulative with the levels below it.

5 The Witnessability Model

Figure 2 summarizes the model: six cumulative levels, each tied to a witness boundary, with the Boundary Ceiling bounding the highest claim a witness can honestly make.

Levels are boundaries, not maturity. An execution-evidence level is not a maturity, quality, or trust ranking: a higher WL reflects control of a stronger execution-relevant boundary, not a more mature, more secure, or otherwise better system. A WL2 record produced by a system that only mediates invocation is not inferior engineering, and a WL5 claim is not an endorsement of the witness beyond the specific evidence that is independently verified.

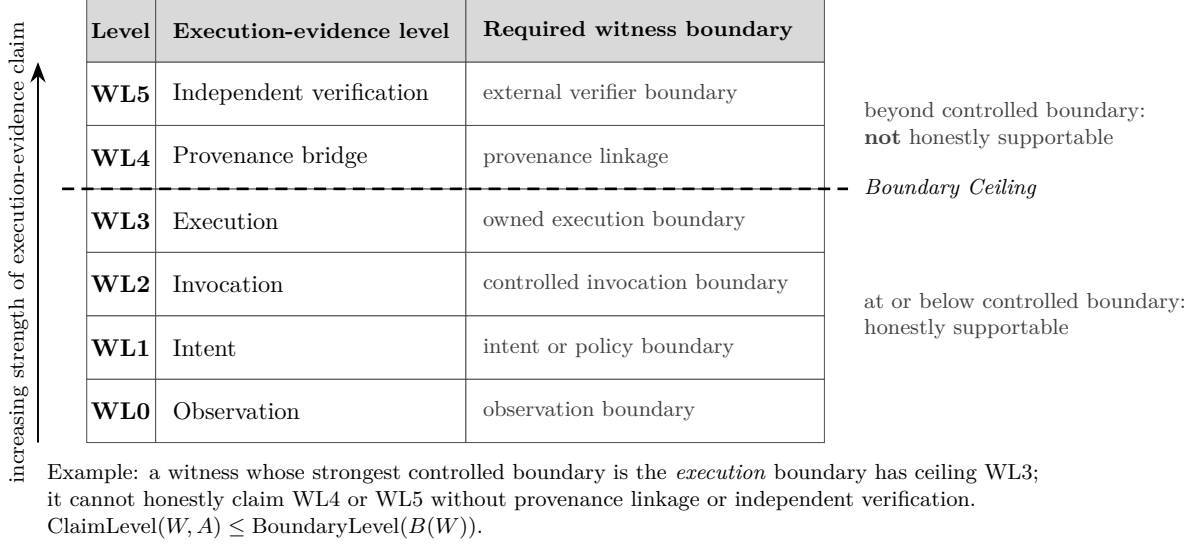


Figure 2: The Witnessability Model. Six cumulative execution-evidence levels (WL0–WL5), each requiring control of a corresponding witness boundary. A witness’s Boundary Ceiling is the highest level supported by the strongest boundary it controls or can independently verify; honest claims lie at or below this ceiling. The dashed line shows an example ceiling for a witness that owns the execution boundary (WL3).

5.1 WL0: Observation

WL0 evidence records that something was observed or reported. It does not prove intent, invocation mediation, execution ownership, provenance linkage, or independent verification.

5.2 WL1: Intent

WL1 evidence records intended, planned, approved, or policy-evaluated action. It proves intent or decision, not necessarily invocation or execution.

5.3 WL2: Invocation

WL2 evidence records that an invocation crossed a controlled boundary. It proves controlled invocation, but not necessarily internal execution of the target system.

5.4 WL3: Execution

WL3 evidence records that the action executed under a controlled execution boundary. It requires execution ownership.

5.5 WL4: Provenance Bridge

WL4 evidence links execution to verifiable origin, such as source commits, build attestations, deployment manifests, artifact digests, SBOMs, or signed configuration records.

5.6 WL5: Independent Verification

WL0–WL4 describe content levels of execution evidence. WL5 describes independent verifiability of a claim at any lower level.

Level	Name	Core claim	Boundary requirement
WL0	Observation	An event was observed or reported.	No execution ownership required.
WL1	Intent	An intended or approved action was recorded.	Control over plan/policy/approval.
WL2	Invocation	A call crossed a controlled invocation boundary.	Control over API/tool/protocol boundary.
WL3	Execution	The action executed under a controlled boundary.	Control over process/activity/runtime.
WL4	Provenance Bridge	Execution is linked to verifiable origin.	WL3 plus provenance linkage.
WL5	Independent Verification	The evidence claim can be verified independently.	External verifier or equivalent mechanism.

Table 4: Execution-evidence levels in the Witnessability Model.

WL5 evidence allows a third party to verify the evidence claim without relying solely on the original operator. WL5 does not necessarily require deterministic replay of the entire execution. Instead, it requires independent verification of the specific evidence claim being made.

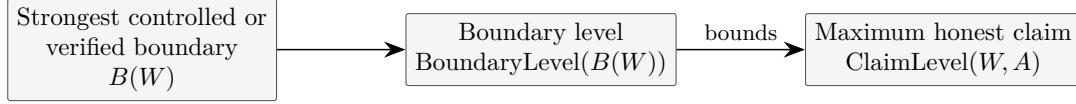
WL5 can be realized through different verification mechanisms that differ in their underlying trust assumptions. A transparency log or public ledger provides independent verification by recording evidence in an append-only structure auditable by third parties, shifting trust to the integrity of the log and its witnesses. A trusted execution environment with remote attestation lets a verifier confirm that evidence was produced inside a measured environment, shifting trust to a hardware root of trust and the attestation chain. A zero-knowledge proof lets a verifier check the validity of a claim while revealing minimal underlying information, shifting trust primarily to cryptographic assumptions. These mechanisms are not mutually exclusive and may be combined; they represent a spectrum of trust assumptions rather than a strict ordering, and the appropriate choice depends on the claim, the adversary model, and applicable privacy constraints.

These mechanisms support a WL5 claim only for the specific claim they independently verify; they do not automatically raise an observation or invocation record to execution evidence.

This reading covers post-hoc verification of a captured claim. A separate case is in-the-loop verification, where an independent verifier participates in the boundary through which the action is authorized, mediated, or executed. In such a configuration, the verifier’s participation is part of the recorded event, but the underlying content level is still determined by the boundary the verifier controls: policy, invocation, execution, provenance, or verification. We leave a full treatment of gate-mediated witnessing to future work.

6 The Boundary Ceiling Principle

The central result of the model is an honesty bound on execution-evidence claims. It constrains what a witness can prove from the boundary it controls or can independently verify; it does not constrain what may have occurred in the world.



Claims above this ceiling add facts the witness cannot establish from its boundary.

Figure 3: Boundary Ceiling Principle as a claim-strength bound. The strongest execution-relevant boundary controlled or independently verified by the witness determines the maximum honest execution-evidence claim.

6.1 Central Result

Principle 1 — Boundary Ceiling Principle. Let W be a witness, A an action, and $B(W)$ the strongest execution-relevant boundary controlled or independently verified by W . Then the strongest execution-evidence claim W can honestly assert about A is bounded by the level supported by $B(W)$:

$$\text{ClaimLevel}(W, A) \leq \text{BoundaryLevel}(B(W)).$$

The bound is an epistemic ceiling: it limits the supportable claim, not the underlying occurrence of the action.

6.2 Rationale

The proof follows from the definition of witnessability as the maximum honest evidence claim supported by a boundary. A witness that only observes an event cannot establish facts requiring execution ownership. A witness that only records intent cannot establish facts requiring invocation. A witness that mediates invocation cannot establish the internal semantics of an external execution it does not own or independently verify. A witness that owns execution can attest to execution results, but independent verification requires additional mechanisms outside sole reliance on the original operator.

Therefore, each execution-evidence level requires a corresponding boundary capability. Claims above that boundary introduce unsupported assertions. Hence the claim level is bounded by the boundary level.

6.3 Corollaries

Observation does not imply execution. Seeing an event is not the same as owning the execution that produced it.

Intent does not imply invocation. A recorded plan or approval does not prove that a call crossed a boundary.

Invocation does not imply execution ownership. A mediated tool call does not prove the internal behavior of the invoked tool unless the witness owns or verifies that execution boundary.

Execution ownership does not imply independent verification. A runtime that owns execution may still be the sole producer and custodian of evidence.

Provenance does not imply runtime action. A build attestation may prove how an artifact was produced, but not that a specific runtime action occurred.

Verifiability does not raise the captured level. A verifiable record of an observation remains an observation; independent verification proves the record is genuine, not that a higher boundary was controlled.

System ceiling and event level differ. A system may be capable of WL3 witnessability while a particular event only achieves WL1 or WL2 if execution results were not captured.

These corollaries define the non-claims of each level — what a record at a given execution-evidence level explicitly does not prove.

6.4 Common False Inferences

Common false inferences.

- Observation $\neq$ Execution.
- Intent $\neq$ Invocation.
- Invocation $\neq$ Execution.
- Execution $\neq$ Provenance.
- Provenance $\neq$ Runtime Action.
- Verification $\neq$ Higher Evidence Level.

7 Witness Boundary Taxonomy

7.1 Observation Boundaries

Observation boundaries capture events without necessarily controlling their cause. Examples include logging agents, metrics collectors, tracing instrumentation, monitoring hooks, and event streams.

7.2 Intent and Policy Boundaries

Intent boundaries record plans, approvals, guardrail decisions, or policy evaluations. Examples include agent planning modules, policy engines, governance layers, approval workflows, and compliance controls.

7.3 Invocation Boundaries

Invocation boundaries mediate calls between actors and tools, services, APIs, or runtimes. Examples include API gateways, tool-calling protocols, MCP clients and servers, function routers, service proxies, and automation connectors.

7.4 Execution Ownership Boundaries

Execution ownership boundaries control actual execution. Examples include process runners, workflow activity workers, CI/CD runners, containers, sandboxes, serverless isolates, virtual machines, agent tool sandboxes, and secure enclaves.

7.5 Provenance Boundaries

Provenance boundaries link execution to origin, source, build, artifact, configuration, dependency, or deployment identity.

7.6 Verification Boundaries

Verification boundaries allow parties outside the original operator to validate evidence claims. Examples include third-party verifiers, transparency logs, public ledgers, reproducible builds, independent witness networks, external audit services, and remote attestation verifiers.

8 Evaluation Method

Each system is classified according to: (1) primary boundary controlled, (2) typical witnessability ceiling, (3) evidence artifacts, and (4) trust assumptions.

When a system supports multiple deployment modes, the classification uses a conservative default: a system is assigned the highest level generally attainable under its typical documented boundary, not the highest level attainable through custom integrations.

9 Canonical Classification Examples

The following examples illustrate how the model classifies common situations and where overclaims usually enter.

9.1 AI Agent Invokes an External SaaS Tool

Claim. An agent claims that an external SaaS tool executed a requested action.

Evidence. The agent runtime may record the planned action, tool name, request parameters, timestamp, and response.

Boundary. The agent controls the planning and invocation boundary, but the SaaS provider controls the tool’s internal execution boundary.

WL ceiling. WL2 for the agent-side invocation record, unless the SaaS execution boundary is independently verified or the SaaS service returns stronger execution evidence.

Common overclaim. Treating a recorded tool call or returned message as proof that the remote tool executed internally as claimed.

9.2 CI/CD Runner Executes a Build

Claim. A CI/CD runner claims that it executed a build command and produced an artifact.

Evidence. The runner may record the command, environment, logs, exit code, artifact digest, and timestamps.

Boundary. The runner controls the process or job boundary in which the build executes.

WL ceiling. WL3 for owned execution evidence; WL4 if the execution is linked to source, artifact, or build provenance; WL5 only for the specific claim independently verified.

Common overclaim. Treating build provenance alone as proof of a later runtime action by the artifact.

9.3 Distributed Tracing Observes a Request

Claim. A tracing system claims that a request crossed a service boundary.

Evidence. Spans, timestamps, service names, request identifiers, latency, status codes, and correlated events.

Boundary. The tracing system observes or instruments request flow but may not own the execution environment that handled the request.

WL ceiling. WL0 for passive observation, rising toward WL2 only where the tracer controls or mediates the invocation boundary.

Common overclaim. Treating a trace of a request as proof that the requested action executed with the asserted internal semantics.

9.4 Adversarial Example: Logged Tool Claim Without Execution

Claim. An agent claims that a tool executed a side effect.

Evidence. The agent log contains a planned tool call and a record asserting success, but the tool never executed.

Boundary. If the agent only records its own plan or self-reported result, the evidence remains at the intent or observation boundary. If it controls a real outbound call boundary, it can support invocation, but not the tool’s internal execution.

WL ceiling. WL1 for self-recorded intent, or WL2 if a controlled invocation actually crossed the tool boundary. It does not reach WL3 unless the witness owns or independently verifies the tool execution boundary.

Common overclaim. Treating internally consistent logs as proof of external execution.

10 Comparative Evaluation

The classifications below are not product ratings or quality rankings; they indicate the typical evidence ceiling attainable under each system’s documented boundary, consistent with the conservative default of Section 8.

Category	Example Systems	Primary Boundary	Typical Ceiling	Notes
Observability [1, 2]	OpenTelemetry, Jaeger, Prometheus	Observation	WL0–WL2	Strong visibility; no inherent execution ownership.
Governance / Guardrails	Policy engines, guardrails	Intent / policy	WL1–WL2	Records decisions and approvals; execution proof requires additional boundary.
Tool Protocols [28, 29]	MCP-based systems	Invocation	WL2	Mediates calls; tool execution depends on server/tool boundary.
Agent Frameworks [24, 31, 30]	LangGraph, AutoGen, OpenAI Agents SDK	Runtime-dependent	WL1–WL3	Ceiling depends on whether framework owns tool execution or only orchestrates.
Workflow Engines [22, 26]	Temporal, Airflow, Prefect, Dagster	Workflow / activity	WL2–WL3	Stronger when activity execution is controlled.
CI/CD Platforms [10, 41]	GitHub Actions, Jenkins, GitLab CI	Runner / process	WL3–WL4	Can execute commands and link to build provenance.
Automation Platforms [42, 43, 44, 45]	Zapier, Workato, n8n	Invocation / workflow	WL2–WL3	SaaS integrations often cross external boundaries.
Serverless Runtimes [36, 37]	Cloudflare Workers, Lambda-like systems	Isolate / runtime	WL3–WL4	Strong execution boundary; provenance linkage depends on deployment evidence.
Coding Agents [33, 35]	Claude Code, Cursor, OpenHands	Sandbox/tool runtime	WL2–WL3	Stronger when execution occurs in owned sandbox.

Category	Example Systems	Primary Boundary	Typical Ceiling	Notes
Supply-Chain Attestation [6, 8]	SLSA, in-toto, Sigstore	Provenance support	WL4 (conditional)	Not a runtime-action witness by itself; can support WL4 only when linked to WL3 runtime evidence; supports independent verification only for the attested claim.
Trusted Execution [38, 39, 40]	TEEs, remote attestation	Measured environment	WL4–WL5 (conditional)	Verifies environment identity/measurement, not action semantics by itself; supports higher levels only when bound to action-level execution evidence.

Table 5: Preliminary classification of contemporary systems.

11 Falsification Attempts and Edge Cases

The model can be stress-tested by asking whether a system produces evidence not covered by WL0–WL5, controls a boundary not covered by the taxonomy, honestly claims execution evidence above its controlled boundary, achieves WL3 without execution ownership, or achieves WL5 without independent verification.

Current stress testing across contemporary systems suggests that systems map onto observation, intent, invocation, execution, provenance, or verification boundaries without requiring a new execution-evidence level. However, governance-decision recorders, physical-world actuation, remote attestation, and cross-boundary agent systems require further investigation.

12 Discussion

12.1 Execution Ownership as Differentiator

The evaluation suggests that execution ownership is the strongest differentiator among systems. Systems with many integrations may still only mediate invocation. Systems with fewer integrations but stronger runtime control may provide stronger witnessability.

12.2 Governance Evidence vs Execution Evidence

Governance evidence answers whether an action was allowed, intended, or compliant. Execution evidence answers what actually executed, under which boundary, with what result. A mature assurance architecture may require both.

12.3 Observability vs Witnessability

Observability answers what the system could see. Witnessability answers what the witness can honestly prove.

12.4 Provenance vs Witnessability

Provenance answers where an artifact, data item, or result came from. Witnessability asks what action can be proven about execution, and how that execution is linked to origin.

12.5 Runtime Verification vs Witnessability

Runtime verification answers whether an execution satisfies a property. Witnessability answers what kind of execution evidence exists for the verification to rely on.

12.6 Why This Is Not Better Logging

Logging is an artifact type. Witnessability is a claim-strength model. The same log record may support different execution-evidence levels depending on where it was produced, which boundary it crossed, and whether it is independently verifiable.

13 Limitations

This paper proposes a conceptual model and taxonomy, positioned alongside vendor-neutral assurance and provenance frameworks rather than as a systems contribution. It does not provide a performance benchmark, security proof, or empirical measurement of all surveyed systems. Systems vary by deployment mode, and the classification should be understood as a typical ceiling, not a universal judgment. Witnessability does not prove that an action was correct, safe, useful, ethical, or aligned with user intent. It only classifies what can be proven about the occurrence and evidence of execution. The model also bounds only what can be honestly claimed when evidence is produced; it does not by itself address evidence suppression—a witness or operator may withhold, omit, or decline to produce a record, and controlling a higher boundary does not compel a record to exist. Detecting or deterring such non-production lies outside the model and requires additional mechanisms such as mandatory logging, transparency requirements, or independent checks.

14 Threats to Validity

Compromised witnesses. A compromised witness may produce false evidence or misrepresent the boundary it controls. The model classifies the claim a boundary can support under honest operation; it does not by itself prove that the witness was uncompromised.

Forged logs. A log can be fabricated or altered. Forgery affects the integrity of the evidence artifact, while witnessability classifies the maximum claim the artifact could honestly support if authentic.

Withheld evidence. A witness may omit evidence or decline to produce a record. Boundary control creates the ability to support a stronger claim; it does not guarantee that the evidence will be generated or disclosed.

Collusion. Multiple parties may coordinate to present misleading evidence. Independent verification reduces some reliance on a single operator, but collusion remains an adversarial risk outside the level taxonomy itself.

Ambiguous boundaries. Some systems blur observation, invocation, and execution ownership, especially when responsibility is split across hosted services and plugins. Such cases should be classified conservatively unless the controlled boundary is explicit.

15 Future Work

Future work should refine methods for identifying execution-relevant boundaries, study physical-world actuation and cross-domain runtime settings, and examine how independent verification mechanisms vary across deployment contexts. Further work may also test whether the WL0–WL5 taxonomy remains stable across additional classes of AI and software execution systems.

16 Conclusion

Modern AI and software systems produce many forms of evidence, but these artifacts are often conflated. Logs, traces, approvals, attestations, workflow histories, and verification reports do not all prove the same thing.

This paper introduced the Witnessability Model, a boundary-based framework for classifying the strength of execution-related evidence. The model distinguishes observation, intent, invocation, execution, provenance linkage, and independent verification. The central result is the Boundary Ceiling Principle: $\text{ClaimLevel}(W, A) \leq \text{BoundaryLevel}(B(W))$. A witness cannot honestly claim evidence above the boundary it controls or can independently verify.

The comparative evaluation suggests that execution ownership, rather than integration breadth or policy sophistication, is the primary determinant of attainable witnessability. Witnessability provides a vocabulary for reasoning about execution evidence independently of any specific product, vendor, or implementation.

Author Note

Disclosure: One author, Mikhail A. Sergeev, has a commercial interest in execution-evidence and runtime-assurance tooling. This paper presents a vendor-neutral conceptual model and does not describe, evaluate, or endorse any specific proprietary implementation.

References

- [1] *OpenTelemetry Documentation*. <https://opentelemetry.io/docs/>. Accessed 2026-06-08. 2026.
- [2] *What is OpenTelemetry?* <https://opentelemetry.io/docs/what-is-opentelemetry/>. Accessed 2026-06-08. 2026.
- [3] César Sánchez et al. “A Survey of Challenges for Runtime Verification from Advanced Application Domains (beyond software)”. In: *Formal Methods in System Design* (2019). DOI: [10.1007/s10703-019-00337-w](https://doi.org/10.1007/s10703-019-00337-w).
- [4] Chana Weil-Kennedy et al. *Runtime Verification of Interactions Using Automata*. 2025. arXiv: [2511.00531](https://arxiv.org/abs/2511.00531) [cs.LO].
- [5] Shuo Ding and Qirun Zhang. “Witnessability of Undecidable Problems”. In: *Proceedings of the ACM on Programming Languages* 7.POPL (2023), pp. 982–1002. DOI: [10.1145/3571227](https://doi.org/10.1145/3571227).
- [6] *SLSA Provenance*. <https://slsa.dev/provenance>. Accessed 2026-06-08. 2026.
- [7] *SLSA: Verifying Artifacts*. <https://slsa.dev/spec/v1.0/verifying-artifacts>. Accessed 2026-06-08. 2026.
- [8] *in-toto Attestation Framework*. <https://github.com/in-toto/attestation>. Accessed 2026-06-08.

- [9] *in-toto and SLSA*. <https://slsa.dev/blog/2023/05/in-toto-and-slsa>. Accessed 2026-06-08. 2023.
- [10] *Using Artifact Attestations to Establish Provenance for Builds*. <https://docs.github.com/actions/security-for-github-actions/using-artifact-attestations/using-artifact-attestations-to-establish-provenance-for-builds>. Accessed 2026-06-08. 2026.
- [11] *GitHub Actions: Artifact Attestations*. <https://docs.github.com/en/actions/concepts/security/artifact-attestations>. Accessed 2026-06-08. 2026.
- [12] Mikhail Anatolievich Sergeev. *PAI-CD: A Constitutional Framework for Authorial Sovereignty in Deployed AI Systems*. <https://doi.org/10.5281/zenodo.19151899>. 2026. DOI: 10.5281/zenodo.19151899.
- [13] Krti Tallam. *A Five-Plane Reference Architecture for Runtime Governance of Production AI Agents*. 2026. arXiv: 2606.12320 [cs.AI]. URL: <https://arxiv.org/abs/2606.12320>.
- [14] Yiqi Wang et al. *From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents*. 2026. arXiv: 2606.04990 [cs.AI].
- [15] Neelmani Vispute. *Reasoning Provenance for Autonomous AI Agents: Structured Behavioral Analytics Beyond State Checkpoints and Execution Traces*. 2026. arXiv: 2603.21692 [cs.AI].
- [16] Adam AlSayyad, Kelvin Yuxiang Huang, and Richik Pal. *AgentTrace: A Structured Logging Framework for Agent System Observability*. 2026. arXiv: 2602.10133 [cs.AI].
- [17] Rohan Sequeira et al. *Agent-Sentry: Bounding LLM Agents via Execution Provenance*. 2026. arXiv: 2603.22868 [cs.CR].
- [18] Peiran Wang, Ying Li, and Yuan Tian. *Aligning Provenance with Authorization: A Dual-Graph Defense for LLM Agents*. 2026. arXiv: 2605.26497 [cs.CR].
- [19] Wenjie Qu et al. *Securing LLM Agents Need Intent-to-Execution Integrity*. 2026. arXiv: 2605.16976 [cs.CR].
- [20] Juan Figuera. *Notarized Agents: Receiver-Attested Confidential Receipts for AI Agent Actions*. 2026. arXiv: 2606.04193 [cs.CR].
- [21] Zheng Chen et al. *Cordon: Semantic Transactions for Tool-Using LLM Agents*. 2026. arXiv: 2606.17573 [cs.OS].
- [22] *Temporal Documentation: Events and Event History*. <https://docs.temporal.io/workflow-execution/event>. Accessed 2026-06-08. 2026.
- [23] *Temporal: Develop Code That Durably Executes*. <https://learn.temporal.io/tutorials/go/background-check/durable-execution/>. Accessed 2026-06-08. 2026.
- [24] *LangGraph Persistence*. <https://docs.langchain.com/oss/python/langgraph/persistence>. Accessed 2026-06-08. 2026.
- [25] *LangGraph Time Travel*. <https://docs.langchain.com/oss/python/langgraph/use-time-travel>. Accessed 2026-06-08. 2026.
- [26] *Apache Airflow Tasks*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/tasks.html>. Accessed 2026-06-08. 2026.
- [27] *Apache Airflow Executor*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/executor/index.html>. Accessed 2026-06-08. 2026.
- [28] *Model Context Protocol Documentation*. <https://modelcontextprotocol.io/docs/getting-started/intro>. Accessed 2026-06-08. 2026.

- [29] *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>. Accessed 2026-06-08. 2024.
- [30] *OpenAI Agents SDK: Tracing*. <https://openai.github.io/openai-agents-python/tracing/>. Accessed 2026-06-08. 2026.
- [31] *Microsoft AutoGen*. <https://github.com/microsoft/autogen>. Accessed 2026-06-08. 2026.
- [32] *AutoGen Multi-agent Conversation Framework*. https://microsoft.github.io/autogen/0.2/docs/Use-Cases/agent_chat/. Accessed 2026-06-08. 2026.
- [33] Xingyao Wang et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. 2024. arXiv: [2407.16741 \[cs.AI\]](#).
- [34] Xingyao Wang et al. *The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents*. 2025. arXiv: [2511.03690 \[cs.SE\]](#).
- [35] Rajesh Kumar et al. *AgentForge: Execution-Grounded Multi-Agent LLM Framework for Autonomous Software Engineering*. 2026. arXiv: [2604.13120 \[cs.SE\]](#).
- [36] *Cloudflare Workers: How Workers Works*. <https://developers.cloudflare.com/workers/reference/how-workers-works/>. Accessed 2026-06-08. 2026.
- [37] *Cloudflare Workers Security Model*. <https://developers.cloudflare.com/workers/reference/security-model/>. Accessed 2026-06-08. 2026.
- [38] Daniel Hugenroth et al. *Attestable Builds: Compiling Verifiable Binaries on Untrusted Systems Using Trusted Execution Environments*. 2025. arXiv: [2505.02521 \[cs.CR\]](#).
- [39] Javier Ron and Martin Monperrus. *Verifiable Provenance of Software Artifacts with Zero-Knowledge Compilation*. 2026. arXiv: [2602.11887 \[cs.CR\]](#).
- [40] Fernando Castillo et al. *An Evidence-driven Protocol for Trustworthy CI Pipelines*. 2026. arXiv: [2605.21089 \[cs.SE\]](#).
- [41] *Jenkins Pipeline: Nodes and Processes*. <https://www.jenkins.io/doc/pipeline/steps/workflow-durable-task-step/>. Accessed 2026-06-08. 2026.
- [42] *Zapier: Automate AI Workflows, Agents, and Apps*. <https://zapier.com/>. Accessed 2026-06-08. 2026.
- [43] *Zapier App Integrations*. <https://zapier.com/apps>. Accessed 2026-06-08. 2026.
- [44] *n8n Documentation: Executions*. <https://docs.n8n.io/workflows/executions/>. Accessed 2026-06-08. 2026.
- [45] *n8n Documentation: Logging*. <https://docs.n8n.io/hosting/logging-monitoring/logging/>. Accessed 2026-06-08. 2026.